

BD Copilot — Security & Architecture Summary (One Page)

Purpose: Support Entra ID / Azure resource provisioning and security review.
Audience: Identity, Azure platform, and information-security teams.
Version: Pilot (Phase 1–2) · August 2026

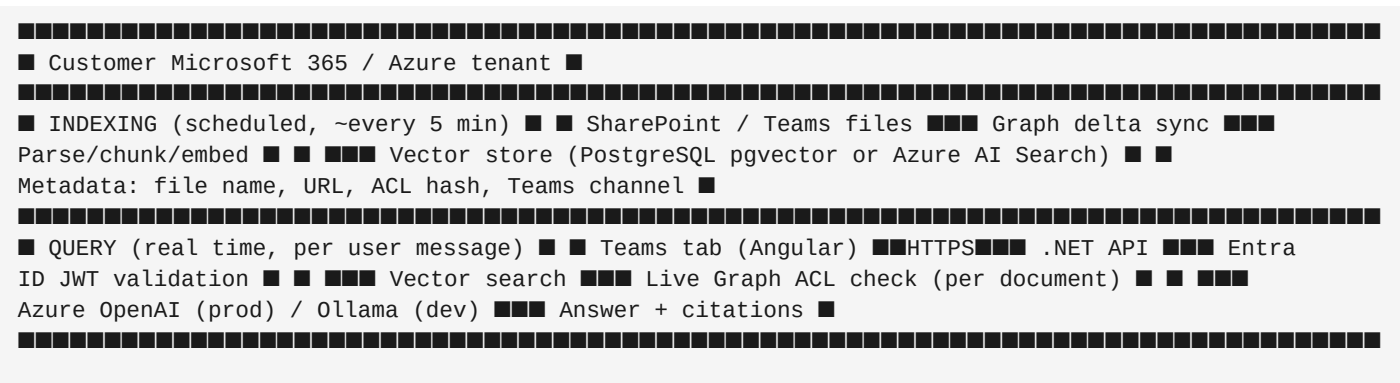
1. What the application does

BD Copilot is a Microsoft Teams tab application for Business Development teams. It provides:

- **AI chat** grounded in approved SharePoint/Teams documents, with citations (file, page, slide).
- **Knowledge search** across indexed content the user is permitted to access.
- **Document generators** (RFP, business case, proposal) that draft from the same indexed corpus.
- **Export workflow** (Word/PPT) with a human-approval step before download.
- **Document library & admin views** for sync health, reindex, access audits, and token usage.

All retrieval is **permission-aware**: the system must not surface content a user cannot already open in SharePoint.

2. High-level architecture



Components

Component	Technology	Role
Teams client	Angular 20 + Teams JS SDK	UI embedded in Teams personal tab
API	ASP.NET Core 10	Orchestration, auth, RAG, generators
Identity	Microsoft Entra ID	SSO + API bearer tokens
Document source	Microsoft Graph	Delta sync from SharePoint libraries

Vector index	PostgreSQL + pgvector (pilot) or Azure AI Search (scale)	Semantic retrieval
LLM	Azure OpenAI (staging/prod) or Ollama (local dev only)	Chat and generation
Blob storage	Azure Blob Storage	Exported documents (time-limited SAS URLs)
Background jobs	Hangfire on PostgreSQL	Incremental document sync
Telemetry	Application Insights (optional)	Errors, latency, token/cost tracking

3. Security model

3.1 Authentication

- Users sign in via **Entra ID** when the app runs inside Teams (SSO via `webApplicationInfo` in the Teams manifest).
- The Angular client sends a **Bearer JWT** to the .NET API on every request.
- The API validates tokens against the registered Entra app (tenant, audience, expiry).
- **Admin endpoints** (reindex, audits, eval) require an Entra app role (e.g. `BdCopilot.Admin`) in production.

3.2 Authorization — document access (security trimming)

This is the primary control for data leakage prevention:

1. **At index time:** Each document stores an ACL fingerprint (`acl_hash`) and Graph drive/item identifiers. Only files the sync service can read (application permission scope) are indexed.
2. **At query time:** After vector search returns candidate chunks, the API calls **live Microsoft Graph permission checks** for the requesting user before any text is sent to the LLM.
3. **Audit trail:** Allow/deny decisions are logged to an `access_audit` table for compliance review.

Design principle: Stale index metadata is never trusted alone. Live Graph ACL is re-evaluated on every query because SharePoint permissions change frequently.

In **local development**, ACL enforcement can be disabled (`AzureAd:EnforceAcl: false`); this is **not** permitted in production.

3.3 Data residency and what leaves the tenant

Data	Stored where	Sent to LLM?
Full SharePoint files	Customer tenant (SharePoint) + chunk text in customer DB/index	Only retrieved chunks (snippets) for grounded prompts

User questions	API logs / App Insights (configurable retention)	Yes — as part of chat/generation prompts
Embeddings	Customer PostgreSQL or Azure AI Search	No
Generated exports	Azure Blob (customer subscription)	N/A — output of generation

Production requirement: Azure OpenAI must be deployed in the **customer's own Azure subscription** (same region/policy as other corporate AI workloads). Local Ollama is for developer machines only and must not be used for production user data.

3.4 Network and secrets

- All client ↔ API traffic over **HTTPS**.
- API ↔ Graph, Azure OpenAI, Blob, and PostgreSQL use TLS.
- Secrets (Graph client secret, OpenAI keys, DB connection strings) stored in **Azure Key Vault** or App Service configuration — not in source control.
- Teams manifest `validDomains` restricted to the deployed app host.

3.5 Logging and governance

- **Access audit:** Per-user document access decisions.
- **Token usage:** Prompt/completion tokens logged per user, team, and operation (when enabled).
- **Generation feedback:** Accept/edit/discard signals for quality improvement (no automatic model training in pilot).
- **Application Insights:** Operational monitoring and cost visibility.

4. Azure / Entra requirements (request checklist)

4.1 Entra ID app registration

Item	Detail
App type	Single-tenant (recommended for pilot)
Redirect URIs	Teams tab URL, MSAL silent/popup URIs as needed
Expose API	Optional scoped access for API (<code>api://<client-id>/access_as_user</code>)
App roles	<code>BdCopilot.Admin</code> for admin console
Certificates/secrets	Client secret or certificate for Graph app-only sync

4.2 Microsoft Graph permissions

Application permissions (background document sync — service principal):

- `Sites.Read.All` or **`Sites.Selected`** (preferred — limit to pilot site(s))
- `Files.Read.All`

Delegated permissions (user context in Teams):

- `User.Read`
- `Sites.Read.All` or **`Sites.Selected`**
- `Files.Read.All`

Admin consent required for all of the above.

4.3 Azure resources

Resource	Purpose
Azure OpenAI	GPT-4o (chat/generation) + text-embedding model (768-dim for pilot index)
App Service or Azure Container Apps	Host .NET API
Azure Database for PostgreSQL (with pgvector) <i>or</i> existing Postgres	Metadata + vector index (pilot)
Azure Blob Storage	Generated document exports
Azure AI Search (Phase 4, optional)	Scaled semantic search with security trimming
Application Insights	Monitoring and token-cost reporting
Key Vault	Secrets management

4.4 Pilot scope

- **One SharePoint site** (pilot team/channel document library) — provide **Site ID** to the project team.
- Prefer **`Sites.Selected`** permission with explicit site grant over tenant-wide `Sites.Read.All` where policy allows.

5. Threat considerations (summary)

Risk	Mitigation
Cross-user document leakage via RAG	Live Graph ACL check on every query; audit logging
Over-broad Graph access	Use <code>Sites.Selected</code> ; pilot one site first

Prompt injection	Retrieved content treated as untrusted context; citations required; no tool execution on SharePoint
Unauthorized API use	Entra JWT required in production; admin role for privileged endpoints
Data sent to wrong LLM endpoint	Ai:Provider locked to Azure OpenAI in prod; no Ollama
Stale or incomplete index	Document library shows sync status; Hangfire recurring sync; manual reindex for admins

6. Deployment phases (security maturity)

Phase	Security posture
Dev (current)	Local Ollama, ACL bypass, no API auth — localhost only
Pilot (target)	Entra SSO, Graph sync on one site, live ACL enforcement, Azure OpenAI in tenant
Production	API auth enforced, admin RBAC, App Insights, Key Vault, optional Azure AI Search

7. Contacts & next steps

Action	Owner
Create Entra app + grant Graph permissions	Identity / Azure team
Provision Azure OpenAI + hosting + Postgres + Blob	Azure platform team
Provide pilot SharePoint Site ID	Business / BD team
Security review / sign-off	Information security
Configure app settings & Teams manifest	BD Copilot project team

For technical wiring details see *backend/Docs/PHASE_WIRING.md*. For in-app architecture diagram see the *Architecture* page in the BD Copilot web app.